02_dataprep_01_2026s1: Preparing Data for Analysis (Modified Titanic)

The file “titanic_to_student.csv” contains the data with 12 columns.

1. **PassengerId**: A unique identifier assigned to each passenger.
2. **Survived**: Indicates whether the passenger survived (1) or did not survive (0).
3. **Pclass**: The passenger's class (1 = 1st class, 2 = 2nd class, 3 = 3rd class).
4. **Name**: The full name of the passenger.
5. **Sex**: The gender of the passenger (male or female).
6. **Age**: The age of the passenger in years.
7. **SibSp**: The number of siblings or spouses the passenger had aboard the Titanic.
8. **Parch**: The number of parents or children the passenger had aboard the Titanic.
9. **Ticket**: The ticket number assigned to the passenger.
10. **Fare**: The fare paid by the passenger.
11. **Cabin**: The cabin number where the passenger stayed (if known).
12. **Embarked:** The port where the passenger embarked (C = Cherbourg; Q = Queenstown; S = Southampton).

Problems

Note: Each problem starts from the original input DataFrame and is evaluated independently. Do not carry over results or modifications from previous problems. Repeat a previous step only when explicitly instructed.

Problem 1: Data Exploration

How many rows are there in the “titanic_to_student.csv”?

Problem 2: Feature Filtering

2.1 Drop variables with missing > 50%

2.2 Check all columns except 'Age' and 'Fare' for flat values, drop the columns where flat value > 70%

From 2.1 and 2.2, how many columns do we have left?

Note:

Ensure missing values are considered in the calculation. If using value_counts(), include dropna=False.

Problem 3: Target Cleaning

Remove all rows with missing values in the target variable "Survived".

How many rows do we have left?

Problem 4: Outlier Handling

- Repeat the feature-filtering steps from Problem 2.
- Create a scikit-learn FunctionTransformer that applies IQR clipping to “Fare”.
- Apply the transformer using fit_transform().
- Replace values outside the IQR boundaries:
    - If value < (Q1 - 1.5IQR), replace it with (Q1 - 1.5IQR).
    - If value > (Q3 + 1.5IQR), replace it with (Q3 + 1.5IQR).

What is the mean of “Fare” after replacing the outliers, rounded to 2 decimal places?

Hint: Use FunctionTransformer, fit_transform(), clip(), and round(_, 2).

Problem 5: Numeric Imputation

- Repeat all feature preprocessing from Problem 4.
- Separate the target variable “Survived” from the input features.
- Build a numeric scikit-learn Pipeline that contains:
    - The IQR clipping transformer from Problem 4.
    - SimpleImputer(strategy="mean").
- Use ColumnTransformer to apply the numeric Pipeline to the numeric input columns.

What is the mean of “Age” after imputing the missing values, rounded to 2 decimal places?

Hint:

- Use Pipeline, FunctionTransformer, SimpleImputer, ColumnTransformer, fit_transform(), and round(_, 2).
- Check *verbose_feature_names_out* if you want output column names without transformer prefixes.

Problem 6: Categorical Encoding

- Repeat all feature preprocessing from Problem 5.
- Separate the target variable “Survived” from the input features.
- Build a categorical Pipeline for “Embarked” that contains:

- SimpleImputer(strategy="most_frequent").
- OneHotEncoder with “C” as the reference category, unknown categories encoded as all zeros, and dense output.

- Combine the numeric and categorical pipelines using ColumnTransformer inside a Pipeline.

What is the mean of “Embarked_Q”, rounded to 2 decimal places?

Hint:

- Use Pipeline, FunctionTransformer, SimpleImputer, OneHotEncoder, and ColumnTransformer. Check drop, handle_unknown, sparse_output, and use round(_, 2).
- Check *verbose_feature_names_out* if you want output column names without transformer prefixes.

Problem 7: Train-Test Splitting

Repeat the target cleaning from Problem 3.

Separate the input features (X) and target (y). Split the data into training and test sets using a 70:30 ratio, random_state=123, and stratify=y.

What is the proportion of survivors in the training data, rounded to 2 decimal places?

Hint: Use dropna(), round(_, 2), and train_test_split() from sklearn.model_selection.

Expected Results

Input	Output
Q1	445
Q2	10
Q3	432
Q4	26.27
Q5	29.14
Q6	0.06
Q7	0.42

Disclaimer: The data used in the example, ‘titanic_to_student.csv’, differs from the data used for scoring.

Template codes

The template code snippet here is the same as the 'student.py' file included in the attachment to the student file downloaded for this assignment.

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer, OneHotEncoder


"""
ASSIGNMENT 2 (STUDENT VERSION)

Each problem starts from the original input DataFrame and is evaluated
independently. Do not carry over results or modifications from previous
problems. Repeat a previous step only when explicitly instructed.
"""


def Q1(df):
    """
    Problem 1: Data Exploration

    How many rows are there in "titanic_to_student.csv"?
    """
    # TODO: Code here
    return None


def Q2(df):
    """
    Problem 2: Feature Filtering

    2.1 Drop variables with more than 50% missing values.
    2.2 Check all columns except "Age" and "Fare" for flat values and
        drop columns where a flat value is more than 70%.

    From 2.1 and 2.2, how many columns remain?

    Ensure missing values are considered in the calculation. If using
    value_counts(), include dropna=False.
    """
    # TODO: Code here
    return None


def Q3(df):
    """
    Problem 3: Target Cleaning
```

```
    Remove all rows with missing values in the target variable "Survived".
    How many rows remain?
    """
    # TODO: Code here
    return None


def Q4(df):
    """
    Problem 4: Outlier Handling

    - Repeat the feature-filtering steps from Problem 2.
    - Create a scikit-learn FunctionTransformer that applies IQR clipping to
      "Fare".
    - Apply the transformer using fit_transform().
    - Replace values outside the IQR boundaries:
      - If value < (Q1 - 1.5IQR), replace it with (Q1 - 1.5IQR).
      - If value > (Q3 + 1.5IQR), replace it with (Q3 + 1.5IQR).

    What is the mean of "Fare" after replacing the outliers, rounded to
    2 decimal places?

    Hint: Use FunctionTransformer, fit_transform(), clip(), and round(_, 2).
    """
    # TODO: Code here
    return None


def Q5(df):
    """
    Problem 5: Numeric Imputation

    - Repeat all feature preprocessing from Problem 4.
    - Separate the target variable "Survived" from the input features.
    - Build a numeric scikit-learn Pipeline that contains:
      - The IQR clipping transformer from Problem 4.
      - SimpleImputer(strategy="mean").
    - Use ColumnTransformer to apply the numeric Pipeline to the numeric input
      columns.

    What is the mean of "Age" after imputing the missing values, rounded to
    2 decimal places?

    Hint: Use Pipeline, FunctionTransformer, SimpleImputer,
    ColumnTransformer, fit_transform(), and round(_, 2).
    """
    # TODO: Code here
    return None


def Q6(df):
    """
    Problem 6: Categorical Encoding

    - Repeat all feature preprocessing from Problem 5.
    - Separate the target variable "Survived" from the input features.
    - Build a categorical Pipeline for "Embarked" that contains:
      - SimpleImputer(strategy="most_frequent").
      - OneHotEncoder with "C" as the reference category, unknown categories
        encoded as all zeros, and dense output.
```

```
    - Combine the numeric and categorical pipelines using ColumnTransformer
      inside a Pipeline.

    What is the mean of "Embarked_Q", rounded to 2 decimal places?

    Hint: Use Pipeline, FunctionTransformer, SimpleImputer, OneHotEncoder,
    and ColumnTransformer. Check drop, handle_unknown, sparse_output, and use
    round(_, 2).
    """
    # TODO: Code here
    return None


def Q7(df):
    """
    Problem 7: Train-Test Splitting

    Repeat the target cleaning from Problem 3.
    Separate the input features (X) and target (y). Split the data into
    training and test sets using a 70:30 ratio, random_state=123, and
    stratify=y.

    What is the proportion of survivors in the training data, rounded to
    2 decimal places?

    Hint: Use dropna(), round(_, 2), and train_test_split() from
    sklearn.model_selection.
    """
    # TODO: Code here
    return None
```